Terminus CertSim Review 2

Oct 20, 2026 · @CJ

Summary

All 10 findings from Review 1 are fixed and covered by tests, but 12 new issues turned up. The worst are that the text processing module no longer works, and that 18 of 35 objectives can be completed without the command succeeding. This review covers commit `e6e5a35` ("implement chmod and stderr redirection", pushed Oct 20, 2026 at 8:23 AM) in cjmit18/terminalpractice.

Check	Result
`npm ci`	Pass
`npm run lint`	Pass, 0 errors
`npm test`	Pass, 18 of 18
`npm run build`	Pass, 446 KB JS (127 KB gzip)
GitHub Actions	New workflow added; run status not visible from this session
Module walkthroughs	All 13 complete, but 1 only by accident (Finding 1)

Beyond rerunning every Review 1 case, this review replayed each module's solution walkthrough the way `ModuleRunner` does and checked which objectives completed.

Fixed since Review 1

Each fix was verified by running the original failing command again.

#	Review 1 finding	Now
1	chmod drops special bits	`chmod 4755` gives `rwsr-xr-x`, `1777` gives `rwxrwxrwt`

#	Review 1 finding	Now
2	Symbolic chmod missing	`u+x,g-r` and `o=r` correct; `chmod zz` gives "invalid mode"
3	sudo redirect allowed	Denied; `echo hi | sudo tee /root/x.txt` and `tee -a` work
4	stderr redirects broken	`2>`, `2>>`, `2>&1`, `2>/dev/null` correct in bash
5	mkdir -p broken	Creates the full path; "File exists" and "No such file" reported correctly
6	Chained errors lost	All errors collected in the engine (but see Finding 3)
7	CMD `&` and pipes	`&` sequences, `findstr`, `/i`, `/v` and `more` work
8	rm edge cases	`rm dir` needs `-r`; `rm -f` silent
9	Supplementary groups	Read from `/etc/group`; `id` shows `27(sudo)`
10	Windows mode 777	Now 644

You also added the CI workflow, a README, and removed the stale `server.js` from the clean script. The chmod and stderr code matches what Review 1 proposed.

New findings and solutions

12 new findings. High means a module or its grading is wrong; Medium means a command or claim is wrong; Low is polish.

#	Finding	Severity	File
1	Text processing module hits Permission denied on every step	High	filesystem.ts
2	18 of 35 objectives grade the typed text, not the result	High	lpiModules.ts, windowsPbqs.ts, modules.ts
3	UI hides stderr whenever there is also stdout	Medium	ModuleRunner.tsx

#	Finding	Severity	File
4	The awk lesson extracts the wrong field	Medium	linuxEngine.ts, lpiModules.ts
5	`chmod -R 755 dir` fails	Medium	linuxEngine.ts
6	`usermod` reports success but does nothing	Medium	linuxEngine.ts
7	CMD ignores stderr redirects (`2>` , `2>nul`)	Medium	windowsEngine.ts
8	README claims features that don't exist	Medium	README.md
9	This review's PDF is committed into `public/` and ships in the build	Medium	public/assets/aistudio
10	PowerShell `Where-Object` filter is ignored	Low	windowsEngine.ts
11	`ls -l file` output and missing `true` / `false`	Low	linuxEngine.ts
12	CI and README target outdated Node versions	Low	ci.yml, README.md

1. Text processing module broken by permissions

Evidence. Module `lpi-103-2-text-processing` asks the student to grep `/var/log/auth.log` . That file is `rw-r-----` owned by `root:adm` , and student is not in `adm` , so all 3 walkthrough steps fail with "grep: /var/log/auth.log: Permission denied". The objectives still complete (see Finding 2), so the student is told they succeeded after seeing only errors. This came from Review 1's permission enforcement working correctly against data that assumed it didn't exist.

Solution. Real Ubuntu puts the first user in `adm` for exactly this reason. Add it to the default `/etc/group` in `createDefaultLinuxFs` :

```
content:
`root:x:0:\nadm:x:4:syslog,student\nsudo:x:27:student\nstudent:x:1000:\nfinan
ce:x:1002:\ndevelopers:x:1003:\nnginx:x:1001:`
```

Tested: with this line the full pipeline exits 0 and prints counts.

2. Objectives grade the typed text

Evidence. 18 of 35 verifiers only check `lastCmd.includes(...)`. Typing `echo grep "Failed password"` completes the grep objective. A failing command completes it too, as Finding 1 shows. The README says verifiers "check real filesystem and system state rather than naive regex string matching", which is true for only 17 of 35.

Solution. Pass the command's result to verifiers, then require success plus evidence in the output:

```
// types/modules.ts
export interface CommandContext { command: string; exitCode: number; stdout:
string; }
verifier: (fs: FileNode, state: SystemState, ctx?: CommandContext) =>
boolean;

// ModuleRunner.tsx: store the last result instead of only the text
setLastResult({ command: rawCmd, exitCode: res.exitCode, stdout: res.stdout
});

// lpiModules.ts, the grep objective
verifier: (_, __, ctx) => !!ctx && ctx.exitCode === 0 &&
/grep/.test(ctx.command) && ctx.stdout.includes('198.51.100.44'),
```

For info commands like `lspci`, `df` or `ipconfig /all`, checking `exitCode === 0` plus the command name is enough. For anything that changes state, prefer checking `fs` or `state`.

3. stderr hidden in the UI

Evidence. `ModuleRunner` builds history with `output: res.stdout || res.stderr`. When a line has both, like `ls nope; echo after`, only stdout shows. So Review 1's Finding 6 is fixed in the engine but invisible to the student.

Solution. Show both, and let `TerminalView` color errors red:

```
output: [res.stdout, res.stderr].filter(Boolean).join('\n'),
errorOutput: res.stderr || undefined, // add to HistoryItem and render in red
```

Apply this in both the bash and Windows branches of `ModuleRunner`, and at lines 181 and 199 of `SandboxMode`, which have the same bug.

4. The awk lesson extracts the wrong field

Evidence. The walkthrough teaches `awk '{print $11}'` to pull IPs. In "Failed password for invalid user admin from 198.51.100.44", field 11 is `admin`, not the IP. With Finding 1 fixed, the pipeline prints `1 198.51.100.44` and `2 admin`, so the lesson's answer is wrong. The sim's awk also only supports a single `$N`, so the real world answer can't be typed.

Solution. Support `$NF` and `$(NF-n)` in awk, then change the walkthrough to `awk '{print $(NF-3)}'` (IP, port, number, ssh2 from the end):

```
const resolveField = (fields: string[], token: string): string => {
  if (token === 'NF') return fields[fields.length - 1] ?? '';
  const m = token.match(/^\(NF-(\d+)\)$/);
  if (m) return fields[fields.length - 1 - Number(m[1])] ?? '';
  const n = Number(token);
  return n === 0 ? fields.join(' ') : fields[n - 1] ?? '';
};
// replace the single $N match with all tokens:
const tokens = [...expr.matchAll(/\$(NF|\(NF-\d+\)|\d+)/g)].map(t => t[1]);
// per line (trim first when using the default whitespace delimiter):
const fields = line.trim().split(delimiter);
out.push(tokens.map(t => resolveField(fields, t)).join(' '));
```

This is also a good exam tip to add: field positions shift when log lines vary, which is why `$NF` exists.

5. chmod with flags before the mode

Evidence. `chmod -R 755 dir` fails with "cannot access '755'", because the mode is always `args[0]`. `-R` is common on the LPI exam. A naive "skip anything starting with a dash" fix would break `chmod -w file`, which is a valid symbolic mode.

Solution. Skip only real option flags, and recurse when `-R` is set:

```
const OPTIONS = new Set(['-R', '-v', '-c', '-f', '--recursive', '--
verbose']);
const recursive = args.includes('-R') || args.includes('--recursive');
const rest = args.filter(a => !OPTIONS.has(a));
const modeArg = rest[0];
const targetPaths = rest.slice(1);

const applyTree = (n: FileNode) => {
  const result = applyChmod(n.permissions, modeArg);
  if (result) { n.permissions = result.sym; n.octal = result.octal; }
  if (recursive && n.children) Object.values(n.children).forEach(applyTree);
};
```

Keep the ownership check per node so `-R` can't change files the user doesn't own.

6. usermod is a stub

Evidence. `sudo usermod -aG developers student` exits 0, but `id` still shows only `student` and `sudo`. The case returns success without touching anything. Now that groups are read from `/etc/group`, it can be real. `tar` has the same problem: it always prints the same two fake lines.

Solution. Edit `/etc/group`. This also lets you teach the classic trap: `-G` without `-a` removes the user from every other group.

```
case 'usermod': {
  if (!isRoot) return fail('usermod: Permission denied.');
  const gi = args.findIndex(a => a === '-aG' || a === '-G');
  const user = args[args.length - 1];
  if (gi === -1 || !args[gi + 1]) return fail('Usage: usermod [-a] -G
GROUP[,GROUP] LOGIN', 2);
  const append = args[gi] === '-aG' || args.includes('-a');
  const wanted = args[gi + 1].split(',');
  const groupFile = getNodeAtPath(activeFs, '/etc/group', false)!;
  const lines = (groupFile.content ?? '').split('\n');
  const missing = wanted.find(g => !lines.some(l => l.startsWith(g + ':')));
  if (missing) return fail(`usermod: group '${missing}' does not exist`, 6);
  groupFile.content = lines.map(line => {
    const parts = line.split(':');
    if (parts.length < 4) return line;
    let members = parts[3] ? parts[3].split(',') : [];
    if (wanted.includes(parts[0])) { if (!members.includes(user))
```

```
members.push(user); }
    else if (!append) members = members.filter(m => m !== user);
    parts[3] = members.join(',');
    return parts.join(':');
  }).join('\n');
  return ok();
}
```

For `tar`, either build a real archive node or return an honest "not supported in this sim" message. A fake success teaches nothing.

7. CMD stderr redirects

Evidence. `dir nope 2>nul` still prints "File Not Found". `type nofile 2> err.txt` creates no file. The parser already extracts `stderrRedirect` for every command; the Windows engine just never reads it. The README lists `2>` support without saying it's bash only.

Solution. Mirror the bash handling after `runInner()` in `executeCmdCommand`, with `nul` as Windows' `/dev/null`:

```
const er = parsed.stderrRedirect;
if (er) {
  if (er.type === 'merge') res = { ...res, stdout: [res.stdout,
res.stderr].filter(Boolean).join('\r\n'), stderr: '' };
  else if (er.target.toLowerCase() === 'nul') res = { ...res, stderr: '' };
  else { writeWindowsFile(res.newFs, normalizePath(er.target, res.newCwd,
true), res.stderr, er.type === '>>'); res = { ...res, stderr: '' }; }
}
```

Pull the existing stdout write logic into `writeWindowsFile` so both redirects share it. Also treat `>nul` as discard for stdout.

8. README accuracy

Evidence. The README is a big improvement, but several claims are wrong, and a recruiter who clones the repo will notice:

- The architecture tree lists `Terminal.tsx`, `LpiLabView.tsx` and `WindowsPbqView.tsx`. None exist; the real files are `TerminalView.tsx`, `ModuleRunner.tsx`, `ModuleSelector.tsx` and the sim components.
- It says "18 test suites" and then lists 19 items.

- It lists Network+ (N10-008/009) and Security+ (SY0-701). The code has only two tracks, `lpi` and `comptia-windows`, and no modules for those exams.
- It lists `route` as a Windows command. `route print` returns "not recognized".
- It says verifiers avoid string matching (Finding 2).
- It says Node 18+. Vite 8 requires Node 20.19+ or 22.12+.

Solution. Generate the tree from the real folder, list the exams the modules actually cover (LPI 101 to 107 topics and A+ Core 2), and remove claims until the feature exists. Add a "Roadmap" section for planned exams instead.

9. Review PDF shipped in the build

Evidence. `public/assets/aistudio/Terminus_CertSim_Code_Review_24846cea.pdf` (386 KB) is committed. Vite copies everything in `public/` into `dist/`, so it's in the production build and would be served at a public URL once deployed.

Solution.

```
git rm --cached
public/assets/aistudio/Terminus_CertSim_Code_Review_24846cea.pdf
echo "public/assets/aistudio/" >> .gitignore
```

If you want reviews in the repo, put them in a `docs/reviews/` folder outside `public/`.

10. PowerShell Where-Object

Evidence. `Get-Service | Where-Object Status -eq Stopped` returns the full list, including Running services. Filtering services is a standard A+ PowerShell task, and the student gets no hint that the filter was ignored.

Solution. Support the two common forms on piped objects: `Where-Object Prop -eq Value` and `Where-Object { $_.Prop -eq 'Value' }`. Start with `-eq`, `-ne` and `-like`. Until then, return an error such as "Where-Object filtering is not supported in this sim" rather than unfiltered output.

11. Small Linux gaps

Evidence.

- `ls -l /etc/passwd` prints `rw-r--r--` without the leading `-` file type character, and size `0`. Directory listings show files with size `4096`, which is a directory size.

- `true` and `false` return "command not found", but they're the standard way to demonstrate `&&` and `||`.
- `echo a > b c` writes `a` instead of `a c`. Bash allows arguments after the redirect target.

Solution. Prefix the file type in the single file branch (`'-' + perm`), use `node.content?.length ?? 0` for file sizes, and add `true` (exit 0) and `false` (exit 1). For the redirect, append the leftover tokens after the target back onto the arguments.

12. Node versions

Evidence. CI uses `node-version: 20`. Node 20 reached end of life in April 2026, and Vite 8 needs at least 20.19. The README says Node 18+, which can't run Vite 8 at all.

Solution. Set CI to `node-version: 22` (or 24), and add an `engines` field so npm warns on old versions:

```
"engines": { "node": ">=20.19.0" }
```

Suggestions

The first two are new and would have caught most of this review automatically. The rest carry over from Review 1 and are still open.

New

1. **Test every module's walkthrough.** Add a test that replays each module's `solutionWalkthrough` like `ModuleRunner` does and asserts every step exits 0 and every objective completes. That one test would have caught Findings 1, 2 and 4. About 25 lines:

```
for (const m of [...LPI_MODULES, ...WINDOWS_PBQS]) {
  test(`walkthrough completes: ${m.id}`, () => {
    let fs = cloneFileSystem(m.initialFs), st =
structuredClone(m.initialState);
    let cwd = m.track === 'lpi' ? '/home/student' : 'C:\\Users\\Student',
dp = false;
    const done = new Set<string>();
    for (const step of m.solutionWalkthrough) for (const cmd of
step.command.split('\n')) {
      const r = m.track === 'lpi' ? executeLinuxLine(cmd, cwd, fs, st)
        : executeWindowsLine(cmd, cwd, fs, st, /^[A-Z][a-z]+-[A-
Z]/.test(cmd) ? 'powershell' : 'cmd', dp);
      assert.equal(r.exitCode, 0, `${m.id}: "${cmd}" failed:
${r.stderr}`);
      ({ newFs: fs, newState: st, newCwd: cwd } = r);
      if ('diskpartActive' in r) dp = !!r.diskpartActive;
      m.objectives.forEach(o => { if (o.verifier(fs, st, { command: cmd,
exitCode: r.exitCode, stdout: r.stdout })) done.add(o.id); });
    }
    assert.deepEqual([...done].sort(), m.objectives.map(o =>
o.id).sort());
  });
}
```

2. **Protect `main` and keep commits focused.** Commit `e6e5a35` is titled "implement chmod and stderr redirection" but also adds CI, a README and a PDF. AI Studio also force pushed earlier. In GitHub settings, require the CI check to pass before merging to `main` and block force pushes. Then use one branch and pull request per change, so each review maps to a clear diff.

Carried over from Review 1

3. **Add `ok()` and `fail()` result helpers.** The repeated return object has grown from 87 to 94 copies in the Linux engine (43 in Windows). The fix code above already assumes these helpers exist.
4. **Replace the switch with a command registry**, one file per command group, so each command can be tested alone and tab completion reads from one list.
5. **Share one chain runner** between the Linux and Windows engines; the `&&`, `||`, `;` loop is still duplicated.

6. **Remove AI Studio leftovers.** `.env.example` still asks for a `GEMINI_API_KEY`, and `metadata.json` still declares a server side Gemini capability. Neither is used.
7. **Explain permission errors.** Show which bit blocked the student, such as "others lack r on /var/log/auth.log". Finding 1 is exactly the case where this would teach something.
8. **Save progress** in `localStorage` (wrapped in try/catch) so completed objectives survive a reload.
9. **Add a timed exam mode** that hides hints and walkthroughs.
10. **Deploy it** to GitHub Pages or Cloudflare Pages, after Finding 9 is fixed so the PDF isn't published with it.

Recommended order of work

Fix grading and the broken module first, since those affect what students learn, then catch up on accuracy and polish.

- [ ] **Session 1, grading:** Findings 1, 2 and 4, plus the walkthrough replay test (Suggestion 1) so they stay fixed.
- [ ] **Session 2, quick wins:** Findings 3, 9 and 12. Each takes a few minutes.
- [ ] **Session 3, commands:** Findings 5, 6, 7, 10 and 11.
- [ ] **Session 4, honesty pass:** Finding 8 (README) and Suggestion 6 (AI Studio leftovers), done after the commands so the README describes what actually works.
- [ ] **Session 5, structure:** Suggestions 2 to 5 (branch protection, helpers, registry, shared chain runner).
- [ ] **Later:** Suggestions 7 to 10 (learning features and deployment).